ĐẠI HỌC QUỐC GIA TP HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

BÁO CÁO BÀI TẬP MÔN HỌC MÁY VÀ ỨNG DỤNG

NCS: Phạm Minh Trí (2580868)

NCS: Nguyễn Nhựt Tấn (2580867)

GVHD: TS. Lê Thành Sách, TS. Nguyễn Quốc Minh

Hồ Chí Minh, Tháng 11 năm 2026

1. Thiết lập môi trường thực nghiệm

Môi trường thực nghiệm: Các bài tập được thực nghiệm trên máy tính Intel Core i7-13700, 16GB Ram, GPU: Nvidia Gerce RTX 3060 Ti (8GB VRam) trên hệ điều hành Ubuntu 24.06 LTS. Sử dụng giao diện jupyter được cấu hình bằng docker theo link github: https://github.com/Kaggle/docker-python.

Thư viện sử dụng gồm các bộ thư viện: torch, matplotlib, numpy, seaborn, torchvision, math, sklearn.

2. Tập dữ liệu

Sử dụng MNIST cho toàn bộ các bài tập E1 đến E3. MNIST gồm ảnh chữ số viết tay từ 0 đến 9, ảnh xám kích thước 28x28, gồm 10 lớp. Trong đó có 60.000 ảnh trong tập huấn luyện và 10.000 ảnh trong tập test.

Phương pháp chia:

Tập huấn luyện (training set) lấy 90% ảnh trong tập train ban đầu, tập kiểm định(validation) lấy 10% ảnh trong tập train ban đầu với chiến lượt trộn(suffle). Riêng tập kiểm tra (test set) được cô lập và chỉ được sử dụng cho mục tiêu kiểm tra.

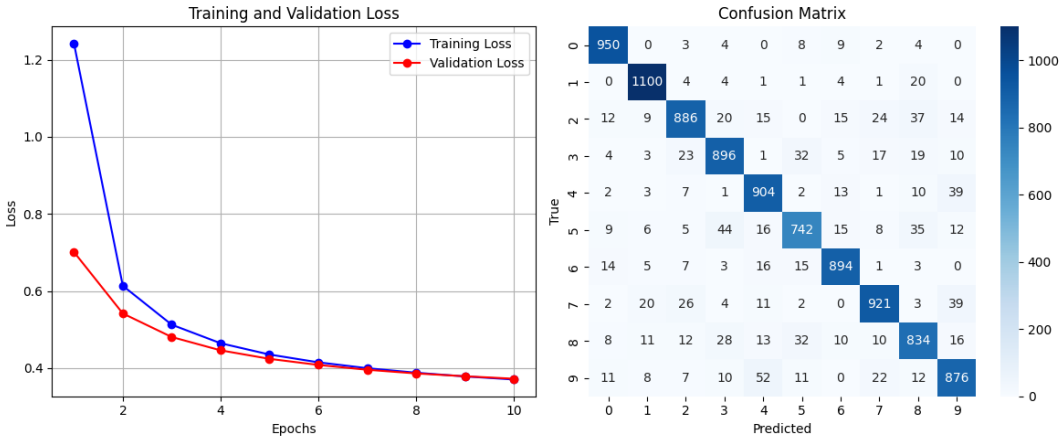
Phương pháp đọc dữ liệu đầu vào và được chuẩn hóa theo giá trị trung bình và độ lệch chuẩn (60.000 ảnh train, chia cho 255) cho kết quả $\mu = 0{,}1307$ và $\sigma = 0{,}3081$.

3. E1 – Phân loại ảnh: Softmax, NLP, CNN

a. Softmax classifier

Ảnh được flatten thành vector $x \in \mathbb{R}^{784}$, ánh xạ tuyến tính rồi qua softmax. Trọng số được khải tạo bằng phương pháp Xavier/Glorot[1] đều trong $[-\frac{\sqrt{6}}{\sqrt{n_{l-1}+n_l}}, +\frac{\sqrt{6}}{\sqrt{n_{l-1}+n_l}}]$, bias bằng 0.

Các tham số được cài đặt batch_size: 64, learning rate (lr): 0.001 và số epochs: 10. Kết quả test sau khi huấn luyện với sai số mô hình loss là 0.3491 và độ chính xác (Accuracy) là 90.03%.



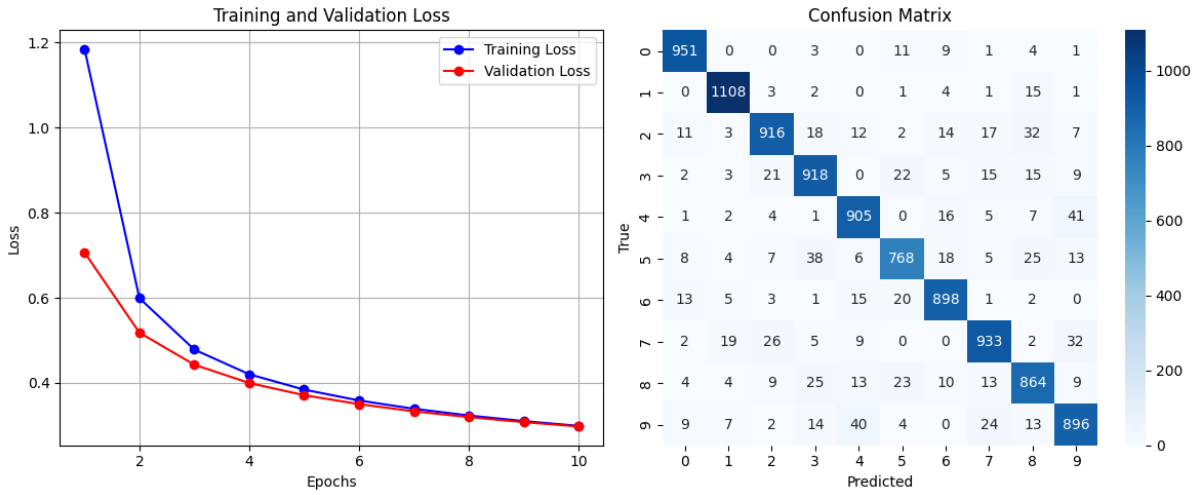
Trong quá trình training từ epoch 1 và 2 ta thấy loss giảm rất mạnh, điều này cho thấy mô hình nhanh chóng học được các features cơ bản của các chữ số. Tuy nhiên, từ epoch 3 đến 6, loss tiếp tục giảm nhưng với tốc độ chậm hơn. Mô hình đang cải hiện khả năng phân loại. Cuối cùng, từ epoch thứ 7 đến 10 hai đường loss của validation và training có xu hướng ổn định và hội tụ.

Khả năng phân loại. Dựa trên ma trận nhầm lẫn thể hiện kết quả phân loại trên 10 lớp từ số 0 đến 9. Hàng 5, cột 3 có 44 ảnh chữ số 5 bị phân loại nhầm với chữ số 3 và hàng 9 cột 4 có 52 ảnh chữ số 9 bị phân loại nhầm thành chữ số 4. Tuy nhiên, nhìn chung đường chéo chính khá lớn và điều này cho thấy mô hình có khả năng nhận dạng tương đối tốt ở tập dữ liệu MNIST.

b. NLP

Đầu vào là vector có 784 chiều được flatten từ ảnh 32x32, sử dụng 2 layer qua hàm kích hoạt ReLU và đi qua softmax. Trọng đó tầng layer 1, trọng số được khởi tạo theo Kaiming He[2] $\sigma = \sqrt{\frac{2}{284}}$ và tầng output, trọng số được khởi tạo theo phương pháp Xavier [1].

Kết quả sau khi chạy 10 epochs có sai số mô hình loss là 0.2833 và độ chính xác là 91.57%



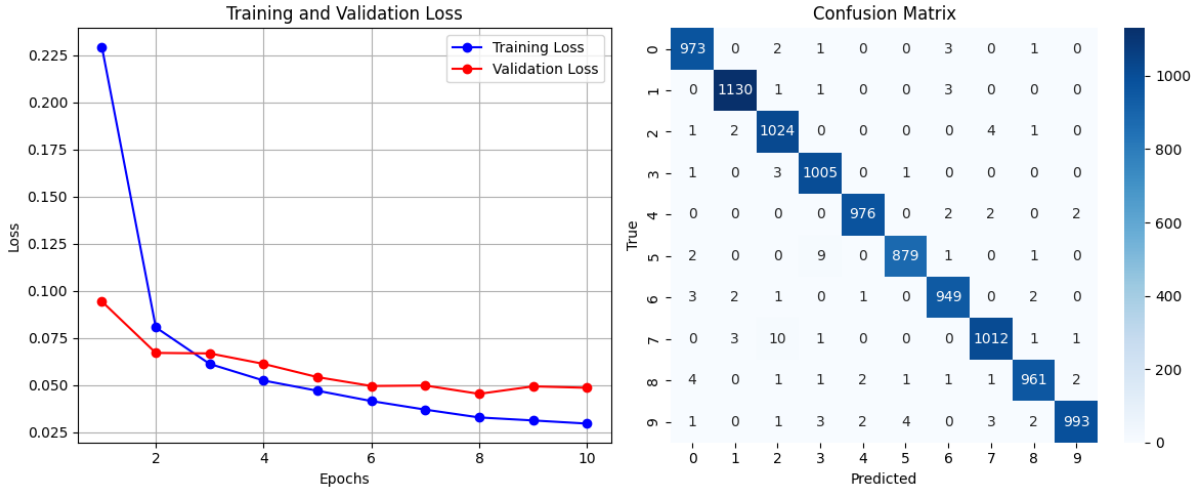
Từ epoch 1 đến epoch 3 loss giảm mạnh, mô hình học được các đặc trưng ban đầu. Trong khi epoch 4 đến 7 loss tiếp tục giảm nhưng chậm dần đến hội tụ ở epoch 8 đến 10 và hội tụ tại 0.30.

Theo ma trận nhầm lẫn class 0 và 1 thì khả năng nhận diện tốt nhất, class 5 có khả năng nhận diện thấp nhất.

c. CNN

Ảnh đầu vào được đưa qua 2 tầng tích chập, sau đó qua lớp dropout (25%) để chống overfitting. Sau đó qua lớp FC.

Tham số chọn bộ tối ưu Adam với learning rate là 0.001, được huấn luyện qua 10 epochs. Kết quả sau khi huấn luyện: lỗi mô hình 0.0354 và độ chính xác mô hình đạt 98.95%.



Thảo luận:

Sau khi thực nghiệm dựa trên cùng chiến lượt phân hoạch dataset, cùng dataset, cùng số lượng epochs và hệ số học. Các tiếp cận như sau:
Phần số lượng tham số.

Mô hình	Cấu trúc	Số tham số	Tối ưu
Softmax	784 → 10	7.850	SGD, η = 0,001
MLP	784 → 128 (ReLU) → 10	101.770	SGD, η = 0,001
CNN	2× [Conv3×3 + ReLU + Pool] → FC	20.490	Adam, lr = 0,001

Số lượng tham số ở mô hình softmax nhỏ nhất. Mô hình đơn giản. Đáng chú ý hơn CNN có ít tham số hơn NLP (khoảng 5 lần). Tuy nhiên kết quả độ chính xác cho thấy mô hình CNN có kết quả tốt nhất 99.02%.

Mô hình	Train Acc	Val Acc	Test Loss	Test acc
Softmax	89,23%	89,37%	0,3491	90,03%
MLP	91,52%	91,40%	0,2885	91,80%
CNN	**99,02%**	**98,65%**	**0,0312**	**99,02%**

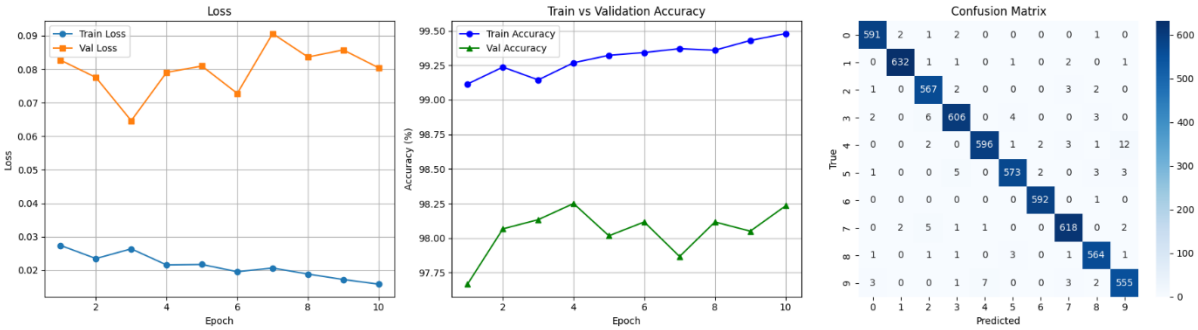
4. E2 – Transformer: MSA và tokenization.

4.1 Hiện thực Multi-Head Self-Attention (MSA) trên tập dữ liệu MNIST.

a. Phương pháp Patch Embeding

Ảnh 28x28 ban đầu được resize lại theo kích thước 32x32. Sau đó sử dụng khối tích chập Con2d có kích thước kernel = 16 và stride = 2 cùng với số kênh đầu vào là 1, số lượng kernel là 64. Phép tích chập này sẽ chia ảnh ra thành các ô vuông(patch) kích thước 16x16 không chồng lên nhau. Như vậy số patch thu được sẽ là 32/16 = 2, tổng cộng tạo ra 2 x 2 = 4 patch. Sau đó được flatten và biến đổi thành chuỗi token có chiều (B, 4, 64). Cuối cùng 1 cls_token có kích thước (1,1,64) sẽ được nối vào đầu chuỗi, kết hợp cùng pos_embedding để cung cấp vị trí cho mô hình.

Kết quả sau khi đánh giá mô hình trên tập dữ liệu kiểm tra, mô hình đạt sai số mô hình là 0.0795 và độ chính xác 98.07%.



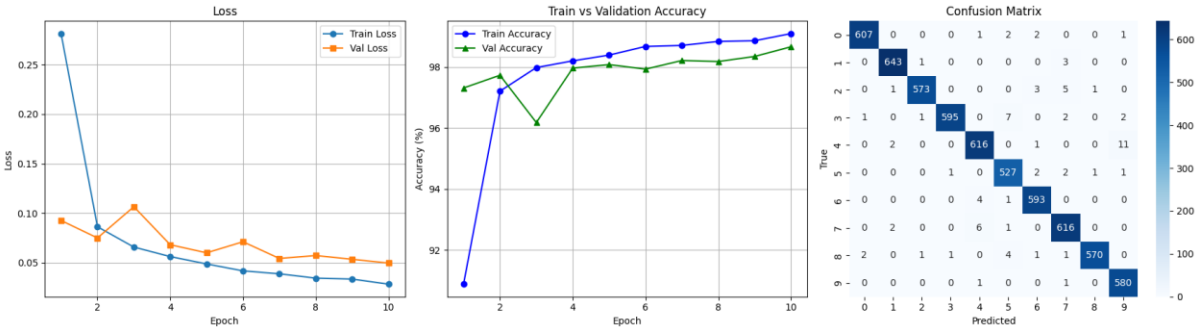
Quá trình training, sai số mô hình giảm đều đặn từ 0.027 xuống 0.016 chứng minh mô hình học rất tốt và tối ưu trên tập huấn luyện. Quá trình kiểm thử (validation) loss dao động từ 0.065 đến 0.09 có xu hướng overfitting.

Ma trận nhầm lẫn thể hiện trên đường chéo chính, mô hình được phân loại chính xác.

b. Phương pháp RowTokenizer

Thay vì xem mỗi ô là một token riêng biệt. Row Tokenizer sẽ gom toàn bộ dữ liệu của các hàng(row) lại thành một chuỗi token duy nhất. Như vậy từ ảnh có kích thước 32x32 sẽ nối 32 pixel từ hàng đầu tiền từ trái qua phải và lần lượt từ trên xuống dưới thành một vector đặc trưng cho hàng đó. Kết quả thay vì chia ảnh thành nhiều patch vuông nhỏ, chúng ta sẽ thu được một chuỗi gồm 32 token tương ứng với 32 hàng của ảnh.

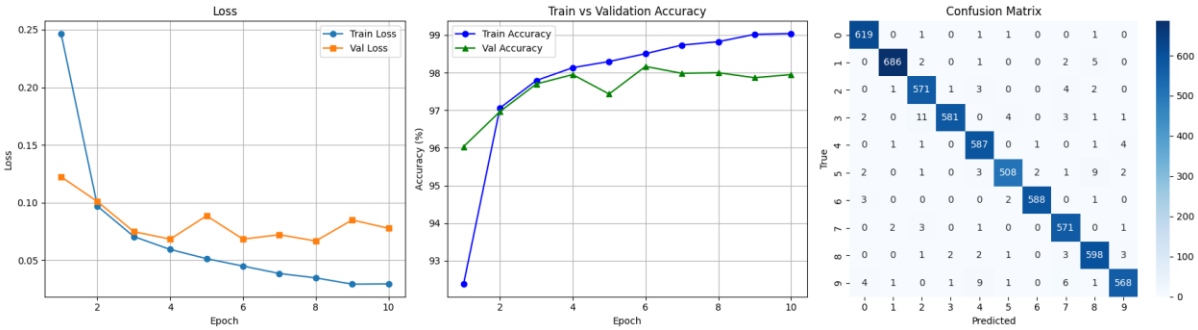
Sau quá trình kiểm tra sau khi huấn luyện mô hình. Mô hình có lỗi sai số đạt 0.04 và độ chính xác 98.85%.



4.2 Sử dụng MSA dùng PyTorch trên tập dữ liệu MNIST.

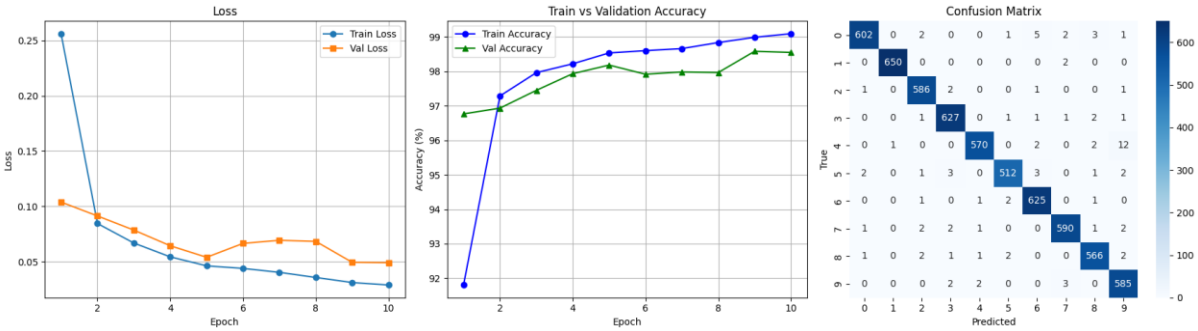
a. Phương pháp PatchEmbedding

Kết quả trên tập kiểm tra sau quá trình huấn luyện mô hình có sai số dự đoán là 0.0746 và độ chính xác là 98.08%



b. Phương pháp RowTokenizer

Kết quả đánh giá trên tập kiểm tra. Mô hình đạt độ chính xác là 98.51%, sai số mô hình đạt 0.0481



Thảo luận:

Kết quả thời gian huấn luyện đến 10 epochs đối với 2 phương pháp embedding và 2 mô hình(tự hiên thực và sử dụng thư viện pytorch). Trong đó, phương pháp MSA sử dụng thư viện pytorch có kết hợp với phương pháp Patch embedding có thời gian tốt nhất(86.4s).

Metric	MSA(Scratch) + Patch	MSA(Scratch) + RowTokenizer	MSA(Pytorch) + Patch	MSA(Pytorch) + RowTokenizer
Time(s)	105	93.8	**86.4**	100.4
Test Loss	0.0688	**0.04**	0.0746	0.0481
Test Acc(%)	98.21	**98.85**	98.08	98.51

Đối với phương pháp embedding, chúng ta có thể thấy phương pháp RowTokenizer có hiệu quả hơn Patch khi cùng một mô hình thì độ chính xác và sai số mô hình tốt hơn.

References

[1] Xavier Glorot, Yoshua Bengio, “Understanding the difficulty of training deep feedforward neural networks,” *Proc. Thirteen. Int. Conf. Artif. Intell. Stat.*, vol. 9:249-256, 2010.

[2] K. He, X. Zhang, S. Ren, and J. Sun, “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification,” in *2015 IEEE International Conference on Computer Vision (ICCV)*, Santiago, Chile: IEEE, Dec. 2015, pp. 1026–1034. doi: 10.1109/ICCV.2015.123.